



Engenharia de Contexto: Fundamentos de LLMs

Tokens, contexto, parâmetros de geração e limites práticos

MÓDULO 1 SEMANA 02 - AULA 1



asCTI.SC

ABOUT ME



- Tech Lead / Senior Machine Learning Engineer
- Mais de 10 anos com IA, visão computacional e NLP
- Experiência em sistemas de ML em produção, edge e cloud
- Atuação com LLMs, RAG, agentes e engenharia de contexto
- Projetos em reconhecimento facial, liveness detection, NLP e IA generativa
- Experiência com Python, PyTorch, Azure, APIs e arquitetura de soluções

CRONOGRAMA

- Segunda (13/04): Fundamentos de LLMs
- Terça (14/04): Estrutura de prompts e boas práticas
- Quinta (16/04): Engenharia de contexto e padrões iniciais

AGENDA

- O que é Engenharia de Contexto
- Como um LLM funciona na prática
- Tokens e tokenização
- Janela de contexto e limites
- Temperatura e parâmetros de geração
- Custos, latência e limitações
- Fechamento e discussão

DEPENDÊNCIA DE CONTEXTO



Query:

Qual foi a causa raiz da falha no serviço de autenticação?

Resposta:

```
{
  "model": "claude-sonnet-4-5",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "Não é possível determinar a causa raiz com precisão sem informações adicionais. Possíveis causas incluem alta carga, falha de rede ou erro interno no serviço."
    }
  ],
  "context": "Não fornecidos"
}
```

DEPENDÊNCIA DE CONTEXTO



Contexto

Service: auth-service

Erro: timeout

Latência média: 1800ms

Número de requisições: alto

CPU: normal

Memória: normal

Retry: falhou após 3 tentativas

Query: Qual foi a causa raiz da falha no serviço de autenticação?

Resposta: Com base nas informações fornecidas, a falha está associada ao aumento de latência no auth-service sob alta carga de requisições, resultando em timeout e falha nas tentativas de retry.

POR QUE O CONTEXTO IMPORTA?

Sem Contexto

Informação insuficiente

Resposta genérica

Baixa capacidade de
inferência

Com Contexto

Informação explícita

Resposta direcionada

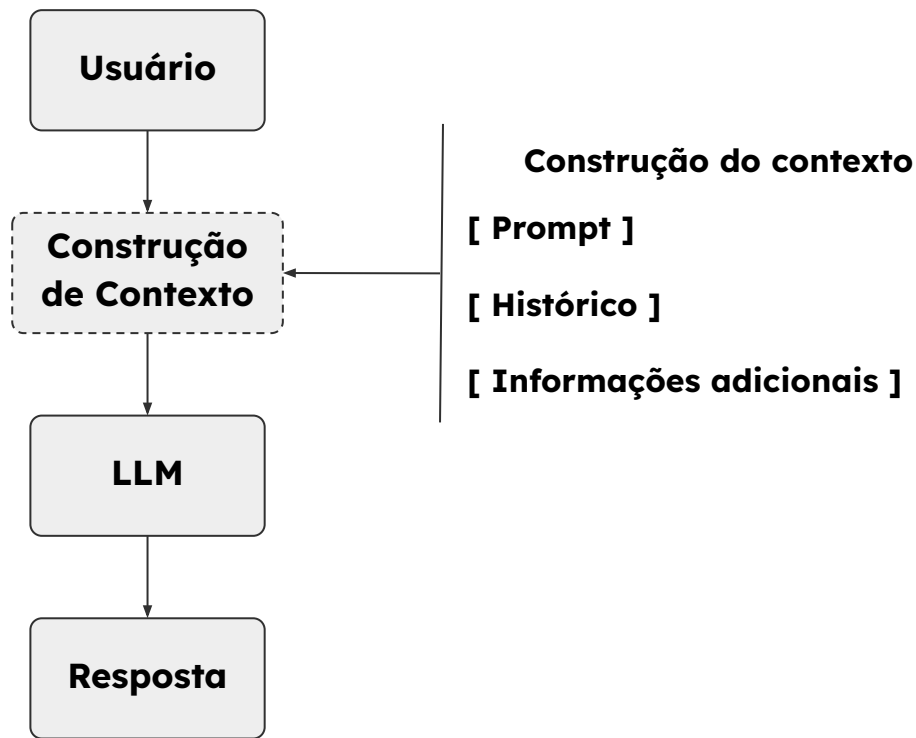
Inferência mais precisa

Mesmo **modelo**, mesma **pergunta**.
O que muda é o **contexto fornecido**.

O QUE É CONTEXTO?

Contexto é a sequência de entrada que condiciona a geração do modelo durante a inferência.

Em termos práticos, é o conjunto de informações disponíveis ao modelo antes da resposta.



COMO O CONTEXTO É CONSTRUÍDO?

[Prompt]

- Instruções (O que fazer)
- Regras (restrições)
- Exemplos (formato esperado)

[Histórico]

- Conversas anteriores
- Estado acumulado

[Informações adicionais]

- Texto fornecido
- Logs
- JSON / Tabelas / Listas

→ Contexto Final → Tokenização

O QUE É UM TOKEN?

Token é a unidade discreta de entrada e saída usada pelo modelo para representar texto durante o processamento.



token $\neq$ palavra

token $\neq$ caractere

token = unidade discreta do vocabulário

POR QUE A TOKENIZAÇÃO É NECESSÁRIA?

A tokenização transforma texto contínuo em unidades manipuláveis pelo modelo.

Texto Natural

- Variável em comprimento
- Ambíguo
- Cheio de símbolos, números, idiomas e estruturas diferentes

“falha no auth-service às 14:32”

Função da Tokenização

- Segmentar o texto em unidades reutilizáveis
- Mapear cada unidade para um índice numérico
- Permitir *lookup* em embeddings
- Tornar viável o processamento sequencial

[falha] [no] [auth] [-] [service] [às] [14] [:] [32]

[3812, 221, 9045, 852, 1469, 874, ...]

TOKEN NÃO É PALAVRA, NEM DIVISÃO UNIFORME

A mesma entrada pode ser segmentada de formas diferentes:

Exemplo 1	Exemplo 2	Exemplo 3
authentication [auth] [entication] ou [authentication] ou [authentic] [ation]	JSON.parse(user_id) [JSON] [.] [parse] [(] [user] [_id] [)]	inconstitucionalissimamente [in][constitucional][issima][mente] ou [inconstitu][cional][issima][mente] ou [in][const][itucion][al][issima][mente]

PRINCIPAIS TÉCNICAS DE TOKENIZAÇÃO

Word-based tokenization

Cada palavra inteira é tratada como uma unidade do vocabulário.

Vantagens

- Segmentação intuitiva
- Fácil interpretação

Limitações

- Vocabulário muito grande
- Problema com palavras fora do vocabulário (OOV)

Exemplo

authentication failed in service

[authentication] [failed] [in] [service]

inconstitucionalissimamente

[inconstitucionalissimamente]

PRINCIPAIS TÉCNICAS DE TOKENIZAÇÃO

Character-based tokenization

Cada caractere é tratado como uma unidade.

Vantagens

- Cobertura total
- Sem OOV

Limitações

- Sequências muito longas
- Maior custo de processamento

Exemplo

authentication

[a][u][t][h][e][n][t][i][c][a][t][i][o][n]

service

[s][e][r][v][i][c][e]

PRINCIPAIS TÉCNICAS DE TOKENIZAÇÃO

Subword tokenization

O texto é segmentado em subtokens, equilibrando cobertura e eficiência.

Vantagens

- Melhor equilíbrio entre vocabulário e sequência
- Lida bem com palavras raras

Limitações

- Segmentação menos intuitiva
- Varia entre modelos

Exemplo

authentication

[auth] [entication]

ou

[authentication]

ou

[authentic] [ation]

PRINCIPAIS TÉCNICAS DE SUBWORD TOKENIZATION

BPE (Byte Pair Encoding)	WordPiece	Unigram
Combina pares frequentes de símbolos ou subtokens.	Escolhe subtokens com base em ganho estatístico no vocabulário.	Escolhe a segmentação mais provável entre várias possibilidades.
authentication [auth] [entication] ou [authentication] Ponto forte: Simples e eficiente	playing [play] [##ing] authentication [auth] [##entication] ou [authentic] [##ation] Ponto forte: bom tratamento de palavras raras	internationalization [international] [ization] ou [inter] [national] [ization] Ponto forte: abordagem probabilística mais flexível

DIFERENÇAS ENTRE MODELOS E IMPACTO PRÁTICO

O mesmo texto pode gerar quantidades e segmentações diferentes de tokens em modelos diferentes.

- Vocabulários diferentes
- Tokenizers diferentes
- Tratamento diferente de espaços, símbolos e bytes

Modelo A → 8 tokens

[Erro] [no] [auth] [-service] [à] [14] [:] [32]

Impacto

- Custo por requisição
- Uso da janela de contexto
- Latência
- Eficiência em código, JSON e diferentes idiomas

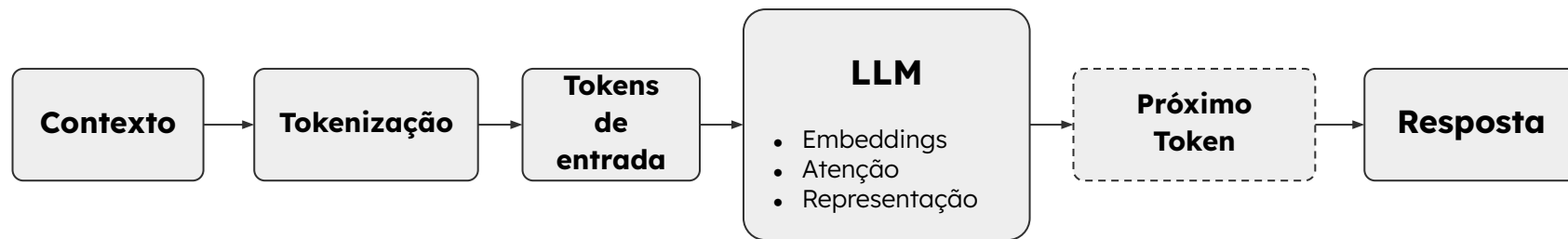
Modelo B → 11 tokens

[Er] [ro] [no] [auth] [-] [service] [à] [s] [14] [:] [32]

VAMOS VER UM POUCO NA PRÁTICA?

TOKENIZAÇÃO

DO TOKEN À GERAÇÃO



Internamente, o modelo transforma **tokens** em **representações numéricas** e usa **atenção** para interpretar o contexto.

A resposta é construída iterativamente, um token por vez.

COMO O MODELO ESCOLHE O PRÓXIMO TOKEN?

[O] [problema] [foi] [causado] [por] [...] ← **Próximo token**

Sequência atual



Representação contextualizada



Logits (Scores para o vocabulário)



Softmax (Probabilidades)

Probabilidades após softmax

[timeout]	0.61
[falha]	0.17
[rede]	0.11
[carga]	0.07
[erro]	0.04

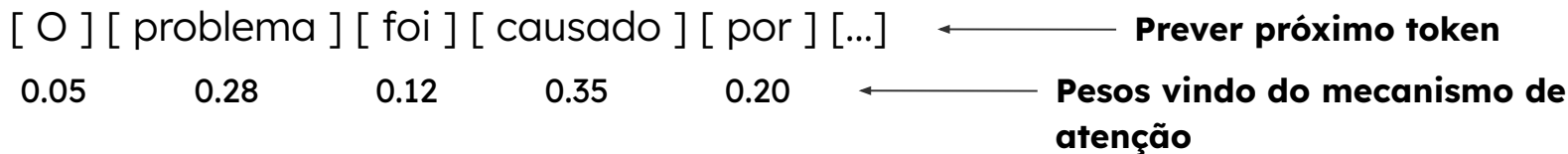
Token selecionado [timeout]

Nova sequência

[O] [problema] [foi] [causado] [por] **[timeout]**

O modelo primeiro produz scores para o vocabulário e depois converte esses scores em probabilidades.

COMO O MODELO ESCOLHE O PRÓXIMO TOKEN?



Combinação ponderada dos tokens anteriores

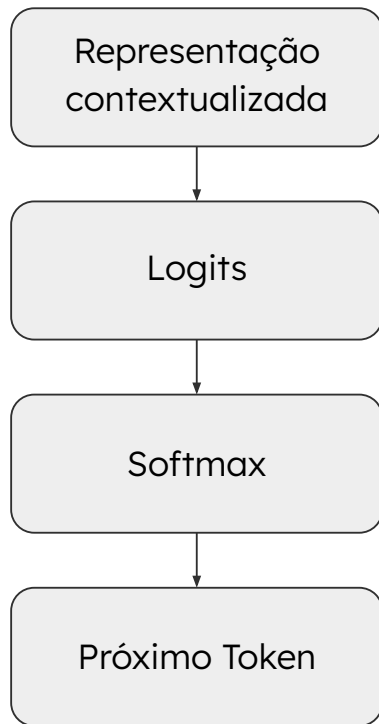
$$\text{contexto} = \Sigma (\text{peso}_i \times \text{representação}(\text{token}_i))$$

$$\text{contexto} = 0.05 * \mathbf{V(O)} + 0.28 * \mathbf{V(problema)} + 0.12 * \mathbf{V(foi)} + 0.35 * \mathbf{V(causado)} + 0.20 * \mathbf{V(por)}$$



Representação contextualizada

DA REPRESENTAÇÃO CONTEXTUALIZADA AO PRÓXIMO TOKEN



Não representa uma palavra específica, mas sim o significado acumulado do contexto.

$$\mathbf{logits} = W \cdot \text{contexto} + b$$

- **contexto** = vetor contextualizado da sequência atual
- **W** = matriz de projeção para o vocabulário
- **b** = bias opcional
- **logits** = scores brutos para os tokens do vocabulário

$$P(\text{token}_i \mid \text{contexto}) = \text{softmax}(\text{logits})_i$$

[timeout]

O token selecionado é anexado à sequência, e o processo se repete até o critério de parada.

O QUE É TEMPERATURA?

Temperatura é um parâmetro que controla o **grau de dispersão da distribuição** de probabilidade dos **próximos tokens**.

Temperatura baixa	Temperatura Alta
• Distribuição mais concentrada • Maior previsibilidade • Menor diversidade • Resultado mais determinístico	• Distribuição mais espalhada • Maior diversidade • Maior risco de desvio • Resultado mais criativo

$$P(\text{token}) = \text{softmax}(\text{logits} / T)$$

COMO A TEMPERATURA ALTERA A DISTRIBUIÇÃO?

Temperatura é um parâmetro que controla o **grau de dispersão da distribuição** de probabilidade dos **próximos tokens**.

Temperatura baixa (0.3)	Base (0.7)	Temperatura Alta (1.5)
timeout → 0.85 falha → 0.08 rede → 0.04 carga → 0.02 erro → 0.01	timeout → 0.61 falha → 0.17 rede → 0.11 carga → 0.07 erro → 0.04	timeout → 0.40 falha → 0.22 rede → 0.16 carga → 0.12 erro → 0.10

Temperatura baixa concentra a distribuição. Temperatura alta espalha a distribuição.

TEMPERATURA BAIXA VS ALTA NA PRÁTICA

Prompt: Explique em uma frase a principal causa da falha no serviço de autenticação.

Temperatura baixa (0.2)	Temperatura alta (1.2)
A falha no serviço de autenticação foi causada por aumento de latência sob alta carga de requisições.	O problema parece estar ligado a uma combinação de sobrecarga, lentidão no serviço de autenticação e falhas nas tentativas de recuperação, o que levou a um comportamento mais instável.

TEMPERATURA E ESTRATÉGIAS DE DECODIFICAÇÃO



Greedy

Escolhe o mais provável

- Mais estável
- Menos diversidade

Resultado: [**timeout**]

Top-K

Mantém apenas os K mais prováveis

- Controla diversidade
- Depende do valor de K

Se **K = 3**

[**timeout**] [**falha**] [**rede**]

Top-P

Mantém o menor conjunto com probabilidade acumulada $\geq p$

- Adaptação dinâmica
- Pode variar mais de caso para caso

Se **P = 90**

[**timeout**] [**falha**] [**rede**]
[**carga**]

TEMPERATURA NÃO RESOLVE FALTA DE CONTEXTO

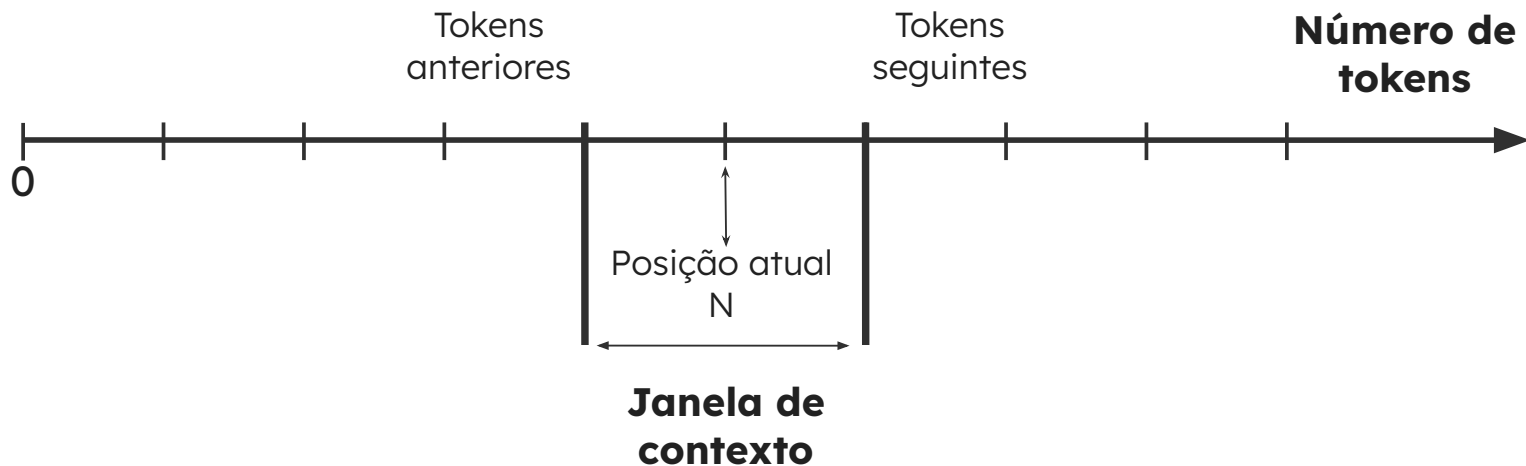
Temperatura controla	Contexto
Diversidade	Evidência
Aleatoriedade	Contexto
Previsibilidade	Precisão factual

Temperatura muda o comportamento da saída, não a qualidade da informação disponível.

VAMOS VER UM POUCO NA PRÁTICA?

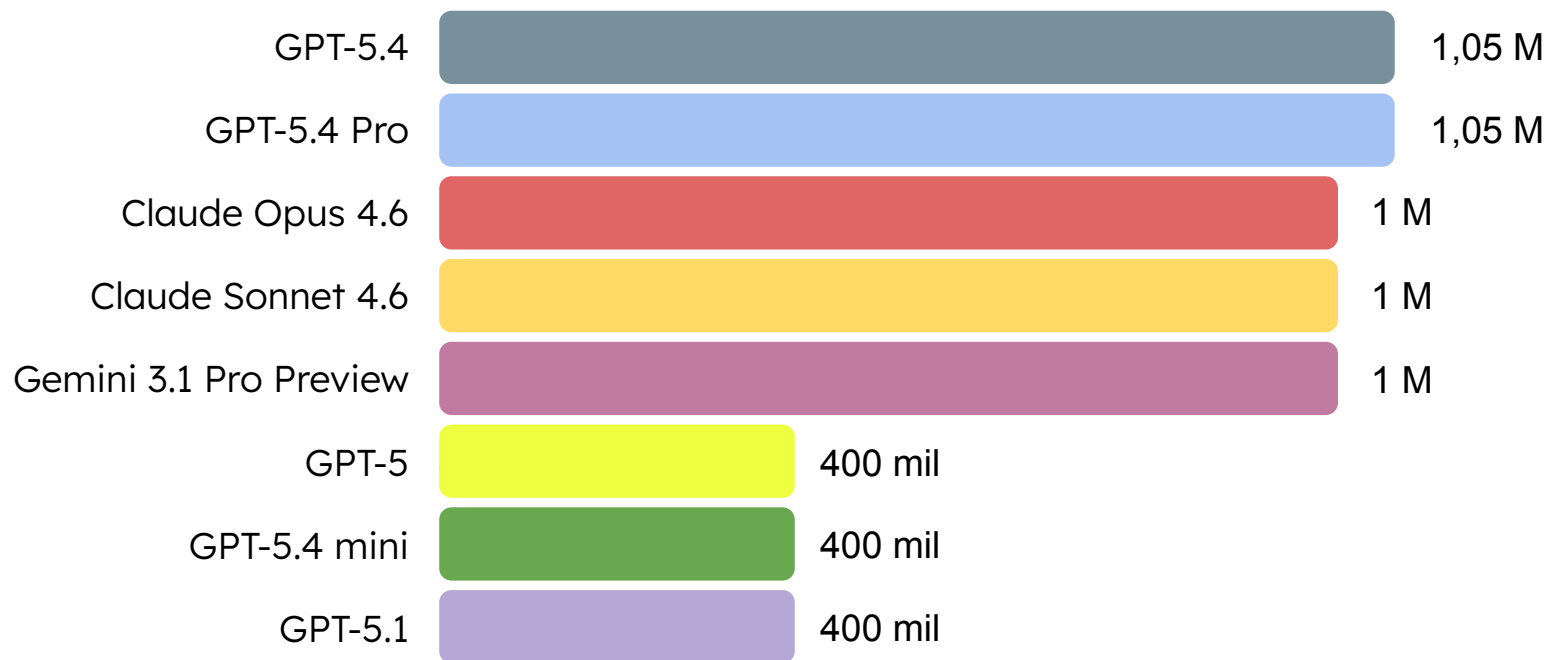
TEMPERATURA

O QUE É UMA JANELA DE CONTEXTO?



Janela de contexto: limite operacional de tokens visíveis ao modelo durante a inferência. Tudo o que o modelo usa para responder precisa caber nesse espaço.

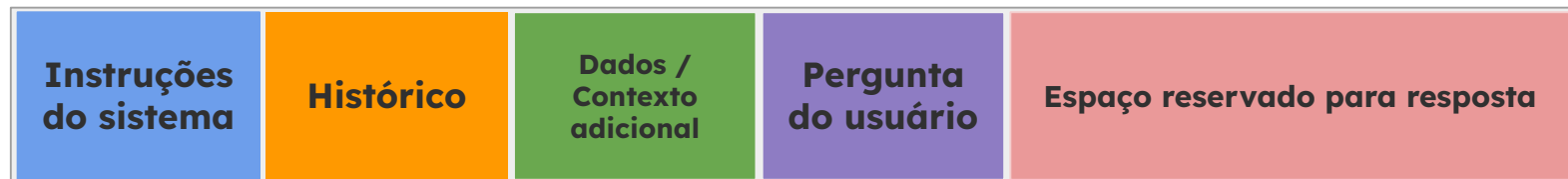
JANELAS DE CONTEXTO EM MODELOS ATUAIS



Valores variam por modelo, versão, endpoint e disponibilidade do produto.

ALOCAÇÃO DA JANELA DE CONTEXTO

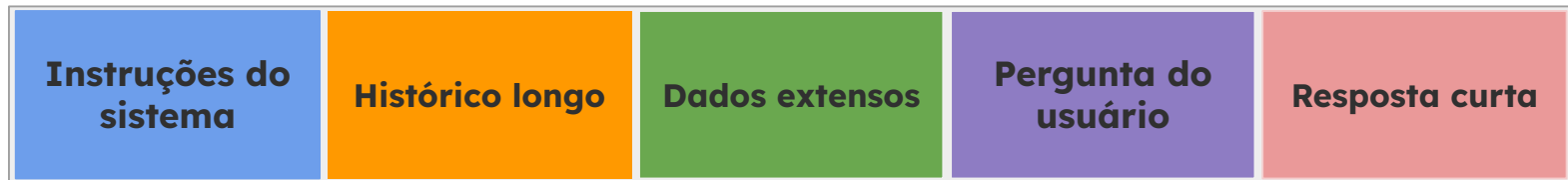
Contexto mais enxuto



0

limite da janela

Contexto mais carregado



0

limite da janela

Mais tokens de entrada reduzem o orçamento disponível para geração

POR QUE JANELAS MAIORES NEM SEMPRE SÃO MELHORES?

1

Sobrecarga informacional

Excesso de
informação pode
diluir o sinal
relevante

2

Perda de foco

Instruções e
evidências
importantes podem
perder prioridade.

3

Ruído e redundância

Contexto maior
frequentemente
aumenta ruído e
redundância.

4

Maior custo / Latência

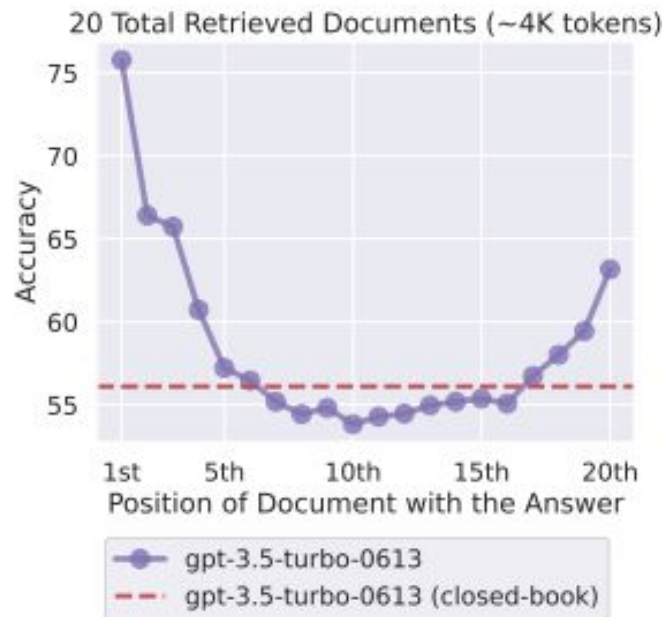
Mais contexto
consome mais
tokens, custo e
tempo de
processamento.

Mais contexto amplia cobertura, mas aumenta o desafio de selecionar o que realmente importa.

LOST IN THE MIDDLE: O PROBLEMA DA POSIÇÃO

- **Início do contexto:** maior destaque
- **Meio do contexto:** maior risco de perda
- **Fim do contexto:** recuperação volta a melhorar

Incluir informação na janela não garante que ela será recuperada com a mesma eficácia em qualquer posição.



fonte:

<https://arxiv.org/pdf/2307.03172>

QUANTIDADE DE CONTEXTO × QUALIDADE DA RESPOSTA

Pouco contexto	Contexto útil	Contexto excessivo
• Pouca evidência • Resposta genérica • Baixa precisão	• Evidência suficiente • Foco na tarefa • Resposta mais precisa	• Mais ruído • Mais redundância • Maior custo e latência

O objetivo não é maximizar contexto, mas fornecer contexto suficiente, relevante e bem organizado.

MESMO CONTEXTO E DUAS FORMAS DE MONTAGEM

Contexto mal projetado	Contexto bem projetado
Logs: [14:32] timeout [14:33] retry failed [14:35] latency spike [14:36] user complaints increased Observação: Esse tipo de falha já apareceu antes em outros serviços. Também houve relatos de comportamento instável em outros horários. A equipe precisa de uma análise. CPU normal. Memória normal. O serviço auth-service estava sob alta carga. Responder de forma objetiva. <u>Qual foi a causa principal da falha?</u>	Objetivo: Identificar a causa principal da falha. Regras: - usar apenas as informações fornecidas - responder em uma frase objetiva Evidências: - auth-service sob alta carga - aumento de latência - timeout - falha no retry - CPU normal - memória normal Pergunta: <u>Qual foi a causa principal da falha?</u>

MESMO CONTEXTO E DUAS FORMAS DE MONTAGEM

Contexto mal projetado	Contexto bem projetado
• Instrução escondida • Evidência diluída • Logs repetidos • Não há separação clara entre dado, regra e objetivo	• Tarefa está explícita • Regras bem destacadas • Evidências agrupadas • Pergunta está isolada

O QUE MUDA QUANDO O CONTEXTO É BEM PROJETADO

Menos ambiguidade

Objetivo, regras e evidências ficam explicitamente separados.

Mais foco

O modelo encontra com mais facilidade o que é prioritário para a tarefa.

Maior previsibilidade

A resposta tende a seguir melhor o formato e a restrição desejados.

Melhor uso da janela

Menos redundância e mais densidade informacional por token.

ESTRUTURA DE CUSTO EM LLMS

Custo total = tokens consumidos × preço do modelo

Tarifa (preço por token)	Consumo (volume)
• Modelo utilizado • Capacidade (mini, pro, etc.) • Custo computacional da inferência	• Tokens de entrada (T_{in}) • Tokens de saída (T_{out}) • Número de requisições

ESTRUTURA DE CUSTO DOS PRINCIPAIS MODELOS

Modelo	Contexto	Custo por 1M tokens
GPT-5.4	1.1M	In: \$2.50 / Cached: \$0.25/ Out: \$15.00
GPT-5.4 mini	400K	In: \$0.75 / Cached: \$0.075 / Out: \$4.50
Claude Opus 4.x	1.0M	In: \$5.00 / Cached: \$0.50 / Out: \$25.00
Claude Sonnet 4.x	1.0M	In: \$3.00 / Cached: \$0.30 / Out: \$15.00
Gemini 3.1 Pro	1.0M	In: \$2.00 / Cached: \$0.20 / Out: \$12.00
Gemini 3 Flash	1.0M	In: \$0.50 / Cached: \$0.05 / Out: \$3.00
GPT-4.1	1.0M	In: \$2.00 / Cached: \$0.50 / Out: \$8.00

fonte: <https://costgoat.com/compare/llm-api>

DEFINIÇÃO DAS TARIFAS

Tamanho do modelo	Arquitetura	Complexidade da Inferência
• Mais parâmetros (mais operações por token) • maior consumo de GPU/TPU • maior custo de inferência	• Número de camadas (depth) • Dimensão dos embeddings • Mecanismos de atenção	Cada token gerado exige: • Atenção sobre toda a sequência • Múltiplas projeções (Q, K, V) • Cálculo de <i>logits</i>
Capacidade do modelo	Infraestrutura e otimizações	Posicionamento de produto
• Reasoning avançado • Multimodalidade • Maior qualidade de resposta	• Quantização (ou ausência dela) • Uso de GPUs vs TPUs • Batching e serving	• Modelos “mini”: Custo otimizado • Modelos “pro”: Máxima capacidade • Modelos “nano”: Alta escala

ESTIMATIVA DE CUSTO EM APLICAÇÕES COM LLM

Teste inicial

- Executar chamadas reais
- Simular cenários de uso

Medição de tokens

- Tokens de entrada (T_{in})
- Tokens de saída (T_{out})
- Uso de logs / API usage

Estimativa por requisição

- **custo** = tokens $\times$ preço do modelo
- considerar **input + output**

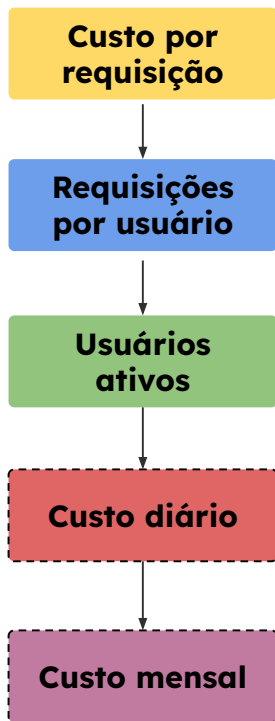
Escala do sistema

- Requisições por usuário
- Usuários ativos
- Frequência de uso

Custo mensal estimado

- Projeção baseada em volume
- Análise de variação (pico vs média)

PROJEÇÃO DE CUSTO EM ESCALA



$$\text{Custo} = (T_{in} \times \$in) + (T_{cached} \times \$cached) + (T_{out} \times \$out)$$

$$\text{Custo diário} = \text{Custo} \times \text{Req/dia} \times \text{Usuários}$$

$$\text{Custo mensal} = \text{Custo} \times \text{Req/dia} \times \text{Usuários} \times 30$$

Legenda:

T_{in} / T_{cached} / T_{out} → tokens (entrada / cache / saída)

$\$in$ / $\$cached$ / $\$out$ → custo por token

Req/dia: requisições por usuário

Usuários: total de usuários

PROJEÇÃO DE CUSTO EM ESCALA (EXEMPLO)

Modelo: GPT-5.4 mini

Input: 2000 tokens

Cached: 1000 tokens

Output: 500 tokens

Preço:

Input: \$0.75 / 1M tokens

Cached: \$0.075 / 1M tokens

Output: \$4.50 / 1M tokens

Input:

$2000 \times (0.75 / 1,000,000) =$
\$0.0015

Cached:

$1000 \times (0.075 / 1,000,000) =$
\$0.000075

Output:

$500 \times (4.50 / 1,000,000) =$
\$0.00225

Total = $0.0015 + 0.000075 +$
 0.00225

Total $\approx$ \$0.0038 por requisição

10 requisições/dia por usuário:

$0.0038 \times 10 = \$0.038$ / dia

1000 usuários:

$0.038 \times 1000 = \$38$ / dia

Custo mensal:

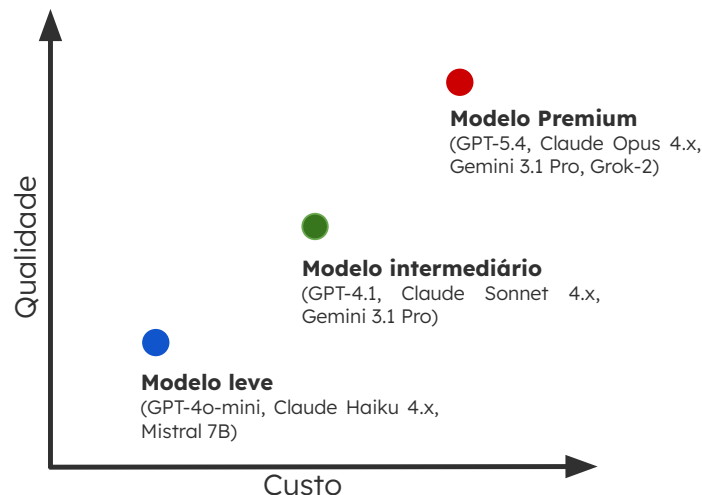
$38 \times 30 \approx \$1,140$ / mês

DECISÃO: CUSTO VS QUALIDADE

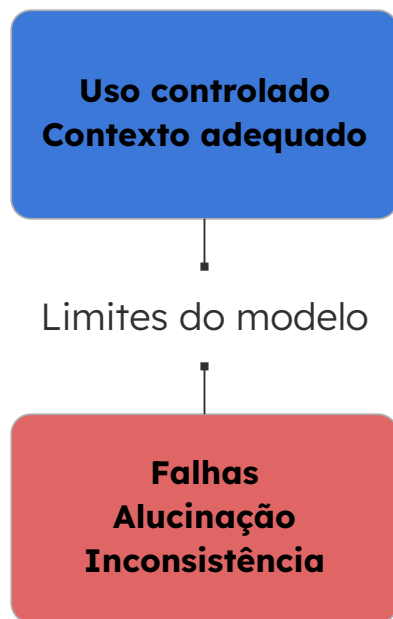
- Qualidade da resposta
- Capacidade de reasoning
- Precisão / robustez
- Custo por requisição

Nem todo problema exige modelo premium

- Escolher o modelo que atende o requisito
- Dentro do orçamento disponível



LIMITES OPERACIONAIS DE LLMS



Limites

- Tamanho finito de contexto
- Custo e latência de inferência
- Comportamento não determinístico
- Dependência do prompt e da entrada

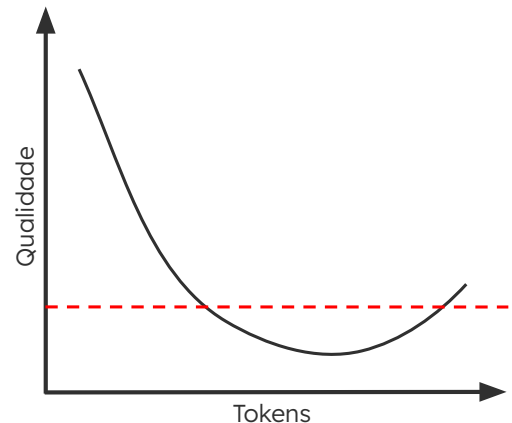
Consequências práticas

- Perda de informação relevante
- Respostas inconsistentes
- Alucinação em ausência de evidência
- Degradação fora do domínio esperado

LIMITES DE CONTEXTO EM LLMS

À medida que o contexto cresce:

- Menos espaço para resposta
- Mais competição entre informações
- Maior chance de ignorar partes relevantes
- Degradação no meio do contexto



0

limite da janela

LIMITES COMPUTACIONAIS EM LLMS

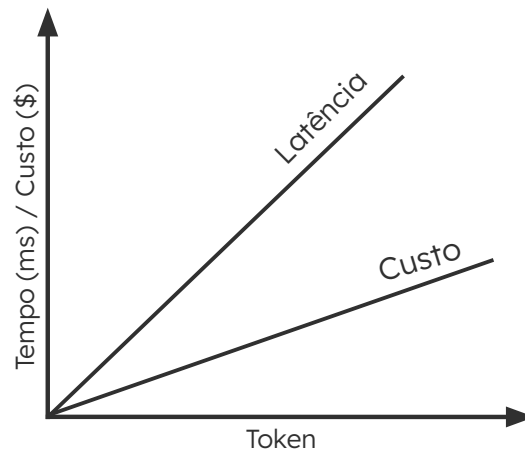
À medida que o número de tokens aumenta:

- Custo cresce linearmente
- Latência cresce com o processamento
- Respostas longas aumentam o tempo de inferência

Base técnica:

- **Prefill:** processamento do contexto
- **Decoding:** geração token a token

Cada token adicional custa tempo e dinheiro



LIMITES DE COMPORTAMENTO EM LLMS

- Alucinação (respostas incorretas com alta confiança)
- Sensibilidade ao prompt
- Inconsistência entre execuções
- Dependência do contexto fornecido

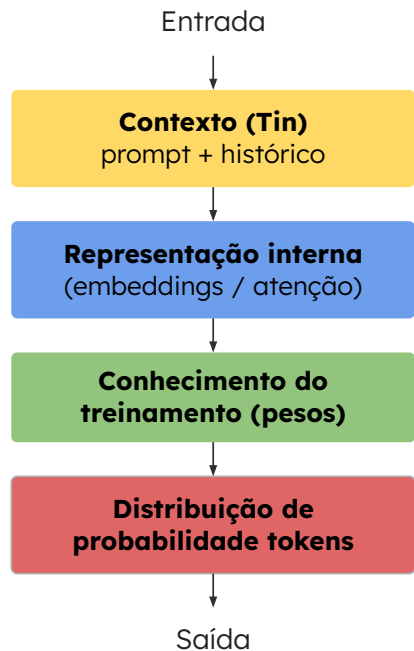
Prompt:

"O que é regularização em machine learning?"

Execução 1: Regularização é uma técnica usada para evitar overfitting, penalizando modelos complexos.

Execução 2: Regularização ajuda o modelo a generalizar melhor, reduzindo o impacto de ruído nos dados de treino.

LIMITES DE CONHECIMENTO EM LLMs



DESAFIO

Você está desenvolvendo um sistema baseado em LLM para atendimento automatizado.

Cada requisição ao sistema utiliza, em média:

- Input: 1500 tokens
- Cached: 500 tokens
- Output: 700 tokens

O modelo utilizado possui os seguintes custos:

- Input: \$1.00 por 1 milhão de tokens
- Cached: \$0.10 por 1 milhão de tokens
- Output: \$5.00 por 1 milhão de tokens

O sistema terá o seguinte uso:

- Cada usuário realiza 8 requisições por dia
- O sistema terá 2.500 usuários ativos por dia

1. Calcule o **custo por requisição**
2. Calcule o **custo diário por usuário**
3. Calcule o **custo diário total do sistema**
4. Calcule o **custo mensal aproximado (30 dias)**



Engenharia de Contexto: Estrutura de Prompts em LLMs

Como definir instruções, objetivos, regras e exemplos para controlar a geração

MÓDULO 1 SEMANA 02 - AULA 2



asCTI.SC

OBJETIVOS DA AULA

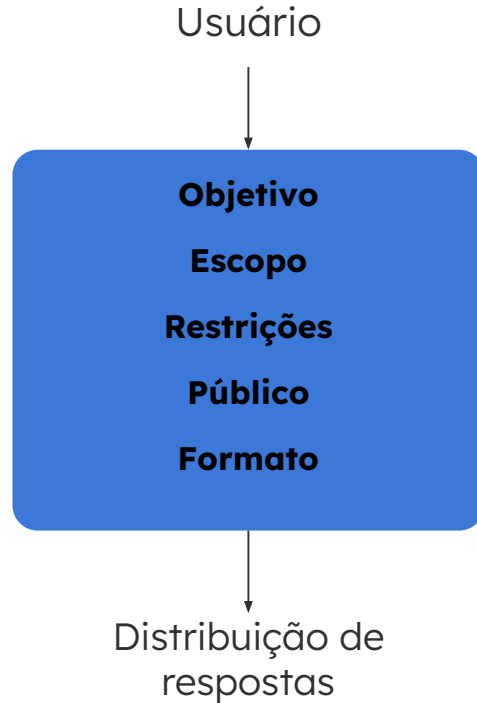
- Estruturar prompts de forma clara
- Reduzir ambiguidade
- Controlar o comportamento do modelo
- Obter respostas mais previsíveis

POR QUE PROMPTS FALHAM?

“Explique machine learning”

Problemas estruturais	Consequências
• Objetivo não especificado • Falta de contexto (nível, domínio) • Ausência de restrições • Formato de saída indefinido	• Respostas genéricas • Alta variabilidade • Baixo controle do resultado

PROMPT COMO ESPECIFICAÇÃO DE TAREFA



Prompt ruim	Prompt estruturado
"Explique IA" Alta variabilidade Resposta genérica	Objetivo + Regras + Formato Resposta controlada Baixa variabilidade

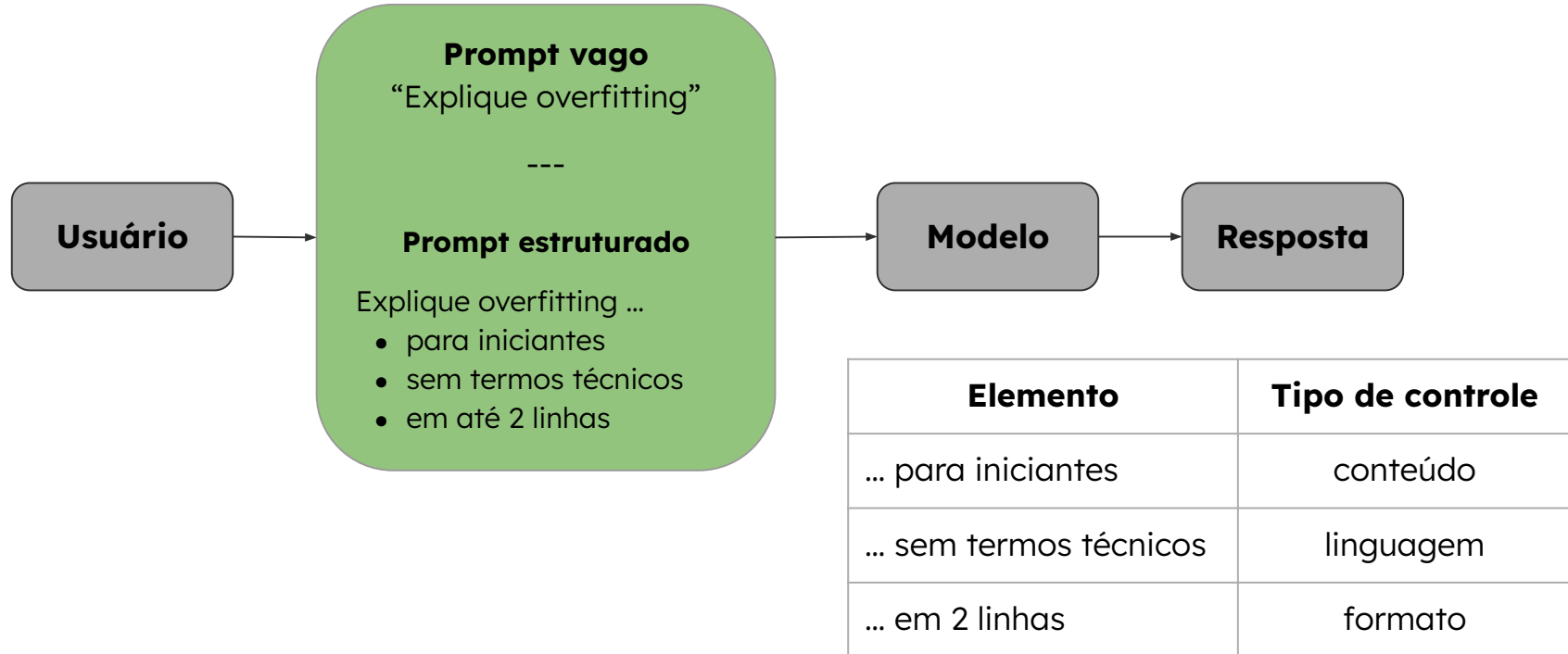
PROMPT COMO MECANISMO DE CONTROLE



O prompt controla

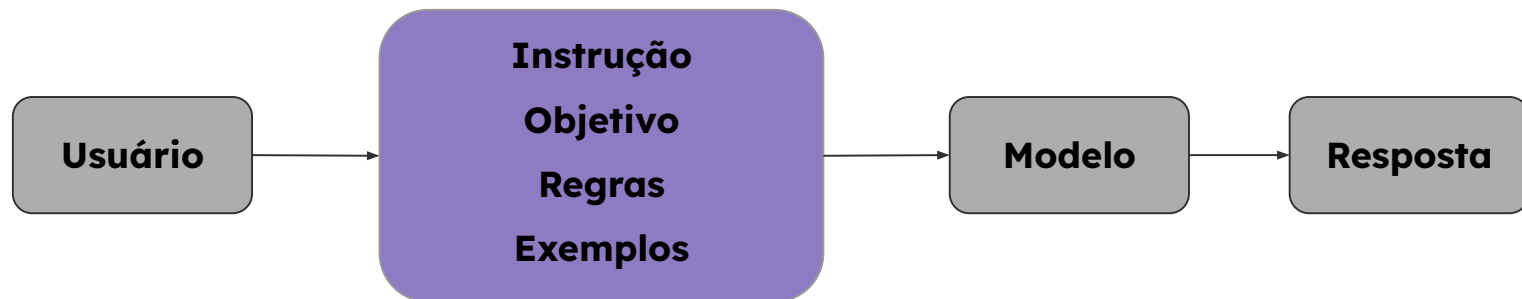
- Conteúdo
- Nível de detalhe
- Estrutura da saída

REDUÇÃO DA AMBIGUIDADE NO PROMPT



ESTRUTURA DE UM PROMPT BEM DEFINIDO

Um prompt eficaz é composto por elementos estruturados que definem o comportamento do modelo



- **Instrução:** define a ação
- **Objetivo:** define o resultado esperado
- **Regras:** definem restrições
- **Exemplos:** induzem padrão de resposta

DEFINIÇÃO DA TAREFA VIA INSTRUÇÃO

Instrução

Explique o conceito de overfitting em machine learning

Objetivo

Regras

Exemplos

Instrução define:

- Tipo de tarefa (task type)
- Estrutura da resposta
- Padrão de geração da resposta

Sem instrução clara, não existe tarefa bem definida.

OBJETIVO: DEFINIÇÃO DO RESULTADO ESPERADO

Instrução

Explique o conceito de overfitting em machine learning

Objetivo

Explicar o conceito de forma acessível para iniciantes, utilizando linguagem simples e exemplos intuitivos

Regras

Exemplos

Objetivo define:

- Público-alvo da resposta
- Nível de profundidade
- Critério de adequação

Exemplos de objetivos:

- Para iniciantes
- Nível técnico avançado
- Focado em aplicação prática

REGRAS: RESTRIÇÕES EXPLÍCITAS NA GERAÇÃO DA RESPOSTA

Instrução

Explique o conceito de overfitting em machine learning

Objetivo

Explicar o conceito de forma acessível para iniciantes, utilizando linguagem simples e exemplos intuitivos

Regras

- Não utilizar termos técnicos
- Limitar a resposta a no máximo 3 linhas
- Não incluir fórmulas matemáticas

Exemplos

Regras definem:

- Restrições de conteúdo
- Limitações de formato
- Restrições de linguagem

Tipos de regras:

- **Conteúdo:** O que pode ou não aparecer
- **Linguagem:** Nível de complexidade
- **Formato:** Tamanho e estrutura

Regras restringem o espaço de respostas possíveis

EXEMPLOS: INDUÇÃO DE PADRÃO VIA FEW-SHOT PROMPTING

Instrução

Explique o conceito de overfitting em machine learning

Objetivo

Explicar o conceito de forma acessível para iniciantes, utilizando linguagem simples e exemplos intuitivos

Regras

- Não utilizar termos técnicos
- Limitar a resposta a no máximo 3 linhas
- Não incluir fórmulas matemáticas

Exemplos

Entrada: conceito simples

Saída:

- definição curta
- explicação direta
- sem termos técnicos

Few-shot prompting:

- Fornece exemplos de entrada/saída
- Induz padrão de resposta
- Reduz necessidade de instruções explícitas

Mecanismo:

- O modelo infere o padrão a partir dos exemplos
- Replica a estrutura observada

Exemplos reduzem ambiguidade estrutural e comportamental

EFEITO DA ESTRUTURA DO PROMPT NO COMPORTAMENTO DO MODELO

Prompt não estruturado	Prompt estruturado
Explique overfitting • Resposta longa • Nível técnico inconsistente • Formato imprevisível	Instrução + Objetivo + Regras + Exemplos • Resposta curta • Linguagem simples • Formato consistente

A estrutura do prompt reduz variabilidade e aumenta previsibilidade da resposta

DECOMPOSIÇÃO DO COMPORTAMENTO DO MODELO

Componente	O que acontece sem ele
Instrução	Ação indefinida
Objetivo	Nível inconsistente
Regras	Perda de controle
Exemplos	Formato variável

Cada componente reduz um tipo específico de incerteza

INSTRUÇÃO: ESPECIFICAÇÃO DA TAREFA

O modelo gera tokens com base em:

$$P(\text{token} \mid \text{contexto})$$

(Probabilidade do próximo token dado o contexto)

A instrução altera essa distribuição ao:

- Aumentar a probabilidade de certos padrões
- Reduzir respostas fora do escopo
- Induzir estruturas específicas

Sinal semântico	Próximo token possível
Explique	“é”, “ocorre”, “refere-se”
Compare	“enquanto”, “por outro lado”
Liste	“1.”, “•”, “-”
Classifique	“positivo”, “negativo”

EFEITO DA AMBIGUIDADE NA DISTRIBUIÇÃO DE TOKENS

Instruções vagas aumentam a dispersão	Instruções claras tornam a distribuição mais concentrada
• Múltiplas interpretações possíveis • Maior variedade de tokens candidatos • Menor previsibilidade Exemplo: Fale sobre overfitting Tokens prováveis: “é”, “exemplo”, “tipos”, “história”, “impacto” ...	• Priorização de respostas alinhadas à tarefa • Maior consistência na saída • Comportamento mais previsível Exemplo: Explique o conceito de overfitting Tokens prováveis: “é”, “ocorre”, “quando”, ...

SEMÂNTICA DOS VERBOS NA INSTRUÇÃO

Tipo de tarefa	Verbo (sinal semântico)	Padrão induzido
Explicação	“Explique”	sequência descritiva
Comparação	“Compare”	estrutura contrastiva
Enumeração	“Liste”	estrutura enumerada
Classificação	“Classifique”	saída categórica
Resumo	“Resuma”	condensação de conteúdo
Análise	“Analise”	decomposição do problema
Avaliação	“Avalie”	julgamento/critério

BOAS PRÁTICAS NA DEFINIÇÃO DE INSTRUÇÕES

Práticas recomendadas

- Use um verbo claro (explique, compare ...)
- Defina uma tarefa principal
- Estructure múltiplas ações (quando necessário)
- Mantenha escopo bem definido
- Alinhe com o tipo de saída desejada

Evite

- Verbos vagos ("**fale sobre**", "**diga algo**")
- Misturar tarefas sem organização
- Escopo amplo demais
- Instruções ambíguas
- Falta de clareza na ação esperada

OBJETIVO COMO ALINHAMENTO DA RESPOSTA

Explique o que é overfitting ...

... para iniciantes

Overfitting é quando um modelo aprende muito bem os dados de treino, mas não funciona bem com novos dados.

... para um engenheiro de ML

Overfitting ocorre quando o modelo **memoriza** o dataset de treino, resultando em baixa generalização para dados **não vistos**.

... para uma criança

É como decorar a prova sem entender, e errar quando a pergunta muda.

O objetivo define como a resposta deve ser construída

INSTRUÇÃO VS OBJETIVO: PAPÉIS DISTINTOS NO PROMPT

INSTRUÇÃO

- Define o que fazer
- Determina a operação
- Controla o tipo de resposta

Define a ação

OBJETIVO

- Define como responder
- Ajusta nível e linguagem
- Alinha ao contexto de uso

Define a forma da resposta

Explique overfitting **para iniciantes**

OBJETIVOS BEM DEFINIDOS: PADRÕES REUTILIZÁVEIS

Objetivo = Público + Formato + Nível de detalhe

Objetivo

Explique **[conceito]**
para **[público]**
em **[formato]**
com **[nível de detalhe]**

Exemplo

Explique o que é **overfitting**
Para **um engenheiro de ML**,
Em formato de **lista**,
com **detalhes técnicos sobre**:

- **generalização**
- **impacto no modelo**
- **relação entre treino e validação.**

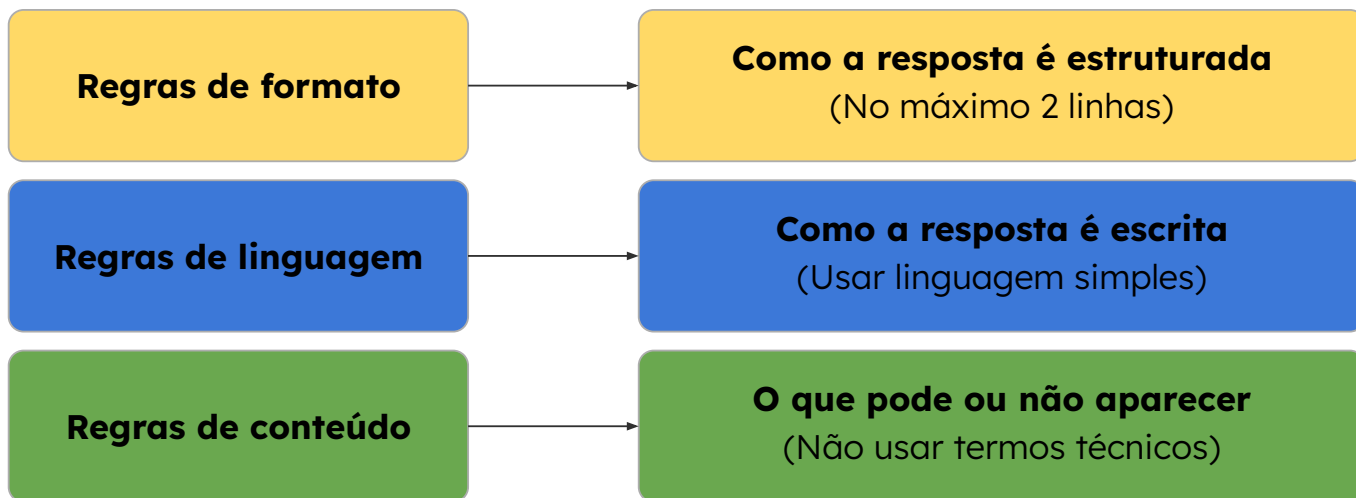
REGRAS COMO CONTROLE DA SAÍDA DO MODELO

“Explique overfitting”

Sem regras	Com regras
Resposta: Overfitting ocorre quando um modelo de machine learning se ajusta excessivamente aos dados de treinamento, capturando ruído e padrões específicos, o que prejudica sua capacidade de generalização para novos dados. Isso pode levar a um desempenho aparentemente alto no treino, mas baixo em dados reais.	Regras: • máximo 2 linhas • não usar termos técnicos • usar linguagem simples • evitar explicações longas Resposta: Overfitting é quando um modelo aprende demais os dados de treino e não funciona bem com novos dados.

TIPOS DE REGRAS E SEU IMPACTO NA RESPOSTA

“Explique overfitting”

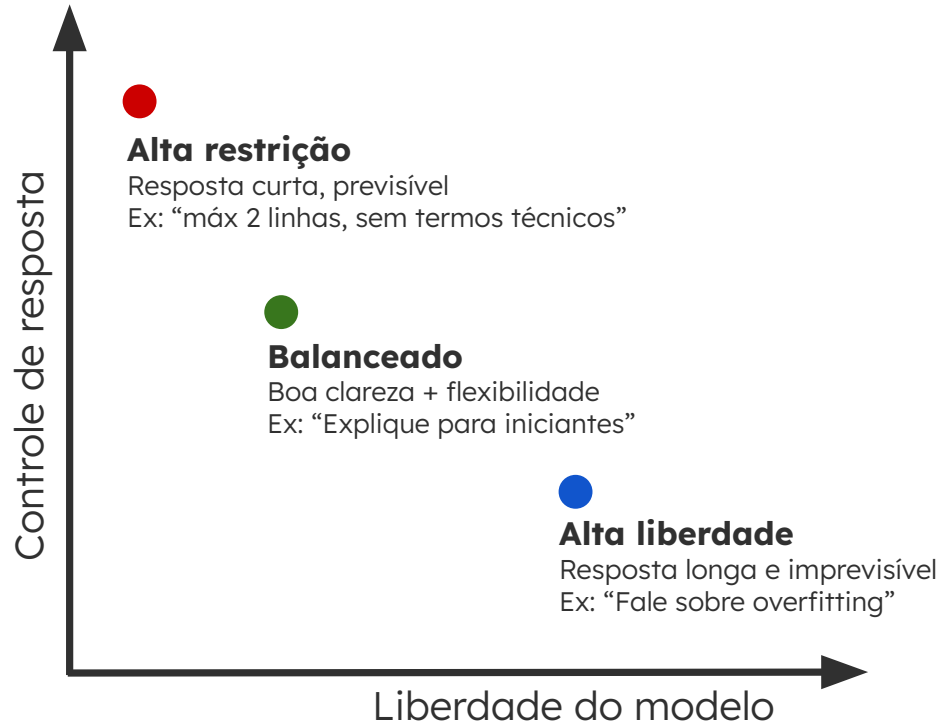


Controle total da resposta exige combinar diferentes tipos de regras

TIPOS DE REGRAS NO PROMPT E SUAS FUNÇÕES

Tipo de regra	O que representa	Exemplo
Conteúdo	O que pode aparecer	não usar termos técnicos
Linguagem	Como escrever	usar linguagem simples
Formato	Estrutura da saída	máximo 2 linhas
Escopo	Limite do conteúdo	focar apenas na definição
Processo / Raciocínio	Como construir a resposta	explicar passo a passo
Saída esperada	Tipo de resposta final	responder apenas “sim” ou “não”
Estilo / Persona	Forma de comportamento	responder como professor

TRADE-OFF: CONTROLE VS LIBERDADE DO MODELO



QUANDO USAR CONTROLE VS LIBERDADE NA PRÁTICA

MAIS CONTROLE

- Sistemas em produção
- Respostas padronizadas
- APIs / outputs estruturados
- Validação de dados
- Classificação / decisão

Ex: classificação de fraude

MAIS LIBERDADE

- Brainstorming
- Geração de ideias
- Escrita criativa
- Exploração de conceitos
- Perguntas abertas

Ex: gerar ideias para um produto

O contexto define o nível ideal de controle

EXEMPLOS COMO DEFINIÇÃO DO PADRÃO DE RESPOSTA

ZERO-SHOT	FEW-SHOT (formato, conteúdo e estrutura da resposta)
Extraia nome e idade “João tem 30 anos” Possíveis respostas: • João tem 30 anos • Ele se chama João e tem 30 anos • Nome: João, idade: 30	Extraia nome e idade Exemplo: “Maria tem 25 anos” → {nome: Maria, idade: 25} Resposta: {nome: João, idade: 30}

Exemplos condicionam o modelo a seguir um padrão específico de geração

QUANDO EXEMPLOS SÃO NECESSÁRIOS

Zero-Shot

Classifique o texto como positivo ou negativo

"Entrega rápida e produto bem embalado"

- Positivo
- Muito positivo
- Avaliação positiva

Variação na saída e Sem padronização

Zero-Shot

Extraia a entidade
"João tem 30 anos"

Possíveis respostas:

- João tem 30 anos
- Ele se chama João e tem 30 anos
- Nome: João, idade: 30

Sem formato rígido

Few-Shot

Classifique como relevante ou irrelevante

Exemplo:

"Erro crítico no sistema":

Relevante

"Ajuste visual na interface":

Irrelevante

Define o critério de decisão

Few-Shot

Extraia a entidade:

Exemplo:

"Maria mora no Rio":

{nome: Maria, cidade: Rio}

"João mora em São Paulo"

{nome: João, cidade: São Paulo}

Define formato da saída

INTEGRANDO TODOS OS ELEMENTOS DO PROMPT

Instrução

Classifique o texto como relevante ou irrelevante

Objetivo

Responder de forma direta, usando apenas a label

Regras

- usar apenas “Relevante” ou “Irrelevante”
- não explicar
- manter resposta em uma palavra

Exemplos

“Erro crítico no sistema”: Relevante

“Ajuste visual na interface” : Irrelevante

Entrada

“Falha na autenticação”

Provável resposta:

Relevante

PROMPT NÃO ESTRUTURADO VS ESTRUTURADO

PROMPT NÃO ESTRUTURADO

Analise o texto abaixo:

“O sistema apresentou falha crítica durante o login”

Resposta:

O sistema teve um problema durante o login e isso pode ser grave. Parece um erro importante, mas depende do contexto.

Análise:

- Resposta descritiva (não decisiva)
- Critério indefinido
- Formato inconsistente

PROMPT ESTRUTURADO

Classifique o texto como ALTA ou BAIXA prioridade

Objetivo

Responder apenas com a prioridade

Regras

- usar apenas “ALTA” ou “BAIXA”
- não explicar

Exemplos

“Erro que impede acesso ao sistema”: ALTA

“Pequeno ajuste visual”: BAIXA

Análise:

- Decisão clara
- Critério definido
- Saída padronizada

O QUE MUDA QUANDO VOCÊ ESTRUTURA UM PROMPT

Sem estrutura:

- Resposta variável
- Interpretação aberta
- Comportamento imprevisível

Com estrutura:

- Resposta consistente
- Decisão clara
- Comportamento controlado

DESAFIOS

Para cada cenário abaixo, construa um prompt estruturado contendo instrução, objetivo, regras, exemplos (quando necessário).

1. Você está criando um tutor de IA para alunos iniciantes.
O sistema deve explicar conceitos técnicos de forma acessível e objetiva.

Sugestão de entrada: “Explique o que é overfitting”

2. Você está desenvolvendo um sistema que analisa feedbacks de usuários.
A saída deve ser exclusivamente:

- POSITIVO
- NEGATIVO

Sugestão de entrada: “O produto chegou rápido e funcionando perfeitamente”



Sistemas de Contexto para LLMs

Blocos estruturados e controle de raciocínio

MÓDULO 1 SEMANA 02 - AULA 3



@SCTI.SC

AGENDA

Bloco 1:

- Engenharia de Contexto
 - Estrutura de entradas para LLM
 - Organização do contexto
 - Controle via estrutura

Bloco 2:

- Direcionamento de Raciocínio
 - Chain-of-Thought
 - Step-by-step prompting
 - Análise crítica das respostas

ENGENHARIA DE CONTEXTO

Prompt funciona quando	Problema em escala
• Contexto é simples • Entrada única • Critério é estável	• Múltiplos inputs • Dados adicionais • Histórico • Critérios dinâmicos

Como organizar tudo isso?

COMPOSIÇÃO DO CONTEXTO EM LLMs

[Camada de Sistema]

- Regras do sistema
- Formato esperado

[Camada de Prompt]

- Instrução
- Objetivo

[Camada de Dados]

- Input do usuário
- Dados externos
- Histórico

[Camada de Raciocínio]

- Guia de raciocínio



PROMPT ESTRUTURADO VS CONTEXTO ESTRUTURADO

Prompt estruturado	Contexto estruturado
# Instrução Classifique o risco da transação # Regras Responder com: Alto Médio Baixo # Exemplo “Compra em país diferente”: Alto # Entrada “Compra de alto valor fora do padrão do usuário”	[Critério] Alto: fora do padrão + alto valor Médio: apenas uma condição Baixo: comportamento usual [Dados] - valor alto - país diferente do histórico [Formato] Alto Médio Baixo

Possível resposta:

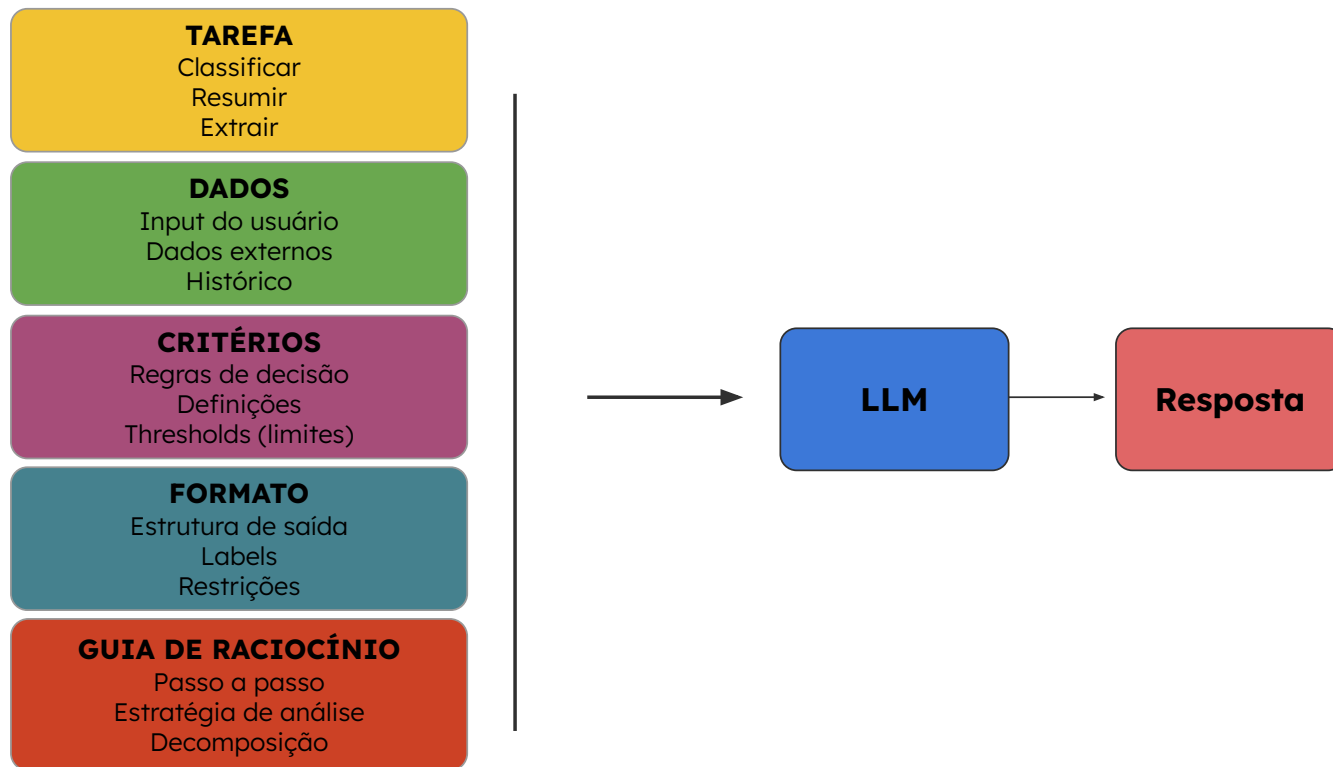
Alto ou Médio

Resposta esperada:

Alto

“Diferença causada pela explicitação do critério”

SEPARAÇÃO DE PAPÉIS NO CONTEXTO



ESTRUTURAÇÃO DO CONTEXTO COMO COMPONENTES REUTILIZÁVEIS

Blocos de informação

[Tarefa]
[Dados]
[Critério]
[Formato]

Função:

- Separar papéis
- Reduzir ambiguidade

Guias de Raciocínio

- Analise passo a passo
- Avalie cada condição
- Justifique antes da resposta

Função:

- Orientar raciocínio
- Reduzir erro lógico

Formulários

Input:

{campo1}, {campo2}

Output:

label única ou JSON
estruturado

Função:

- Padronizar interface
- Garantir consistência

GUIAS DE RACIOCÍNIO: REDUÇÃO DE ERROS POR INFERÊNCIA IMPLÍCITA

SEM GUIA DE RACIOCÍNIO

Pergunta:

Um e-mail contém muitos links e palavras promocionais.
Ele é spam?

Resposta:

Não

Análise:

O modelo não avalia explicitamente sinais de spam e responde com base em associação implícita

COM GUIA DE RACIOCÍNIO

Pergunta:

Um e-mail contém muitos links e palavras promocionais.
Ele é spam?

Guia de raciocínio:

Análise passo a passo:

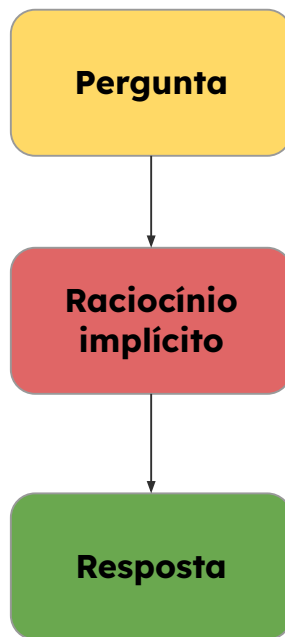
- 1) Identifique características típicas de spam
- 2) Verifique presença dessas características no texto
- 3) Conclua com base na quantidade de sinais

Resposta:

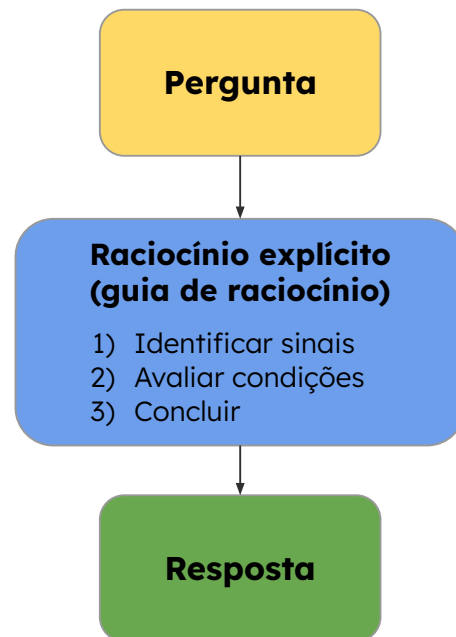
Sim

CHAIN-OF-THOUGHT (COT): RACIOCÍNIO EXPLÍCITO NA GERAÇÃO DE RESPOSTAS

Chain-of-Thought (CoT) é uma técnica de prompting em que o modelo é orientado a gerar ou seguir um raciocínio passo a passo antes de responder.



Maior risco de erro



Maior consistência

APLICANDO CHAIN-OF-THOUGHT EM PROBLEMAS SIMPLES

Sem Chain-of-Thought

Pergunta:

Você tem 3 caixas com 2 bolas cada.
Quantas bolas no total?

Resposta:

5

Com Chain-of-Thought

Pergunta:

Você tem 3 caixas com 2 bolas cada.
Quantas bolas no total?

Raciocínio (passo a passo):

- Cada caixa tem 2 bolas
- 3 caixas: $3 \times 2 = 6$

Resposta:

6

“O ganho não vem da informação, vem da organização do raciocínio.”

APRENDENDO POR EXEMPLOS: FEW-SHOT CHAIN-OF-THOUGHT

Exemplo:

Pergunta:

2 caixas com 3 bolas cada

Raciocínio:

- Cada caixa tem 3 bolas
- $2 \times 3 = 6$

Resposta:

6

Agora responda:

Pergunta:

4 caixas com 2 bolas cada

Resposta:

8

Exemplo:

Pergunta:

Número maior que 10 é alto.
15 é alto?

Raciocínio:

- Regra: $>10 \rightarrow$ alto
- $15 > 10$
- Logo, é alto

Resposta:

Sim

Agora responda:

Pergunta:

Número maior que 10 é alto.
8 é alto?

Resposta:

Não

QUANDO O RACIOCÍNIO SE TORNA ESSENCIAL

Sem Chain-of-Thought

Pergunta:

Se valor > 100 e fora do país: risco alto.
Compra de 150 no país é alto?

Resposta:

Sim

Com Chain-of-Thought

Pergunta:

Se valor > 100 e fora do país: **risco alto**.
Compra de 150 no país é alto?

Raciocínio:

- Valor 150 > 100
- Não está fora do país
- Nem todas as condições atendidas
- Logo, não é risco alto

Resposta:

Não

“Erros acontecem quando o modelo ignora condições.”

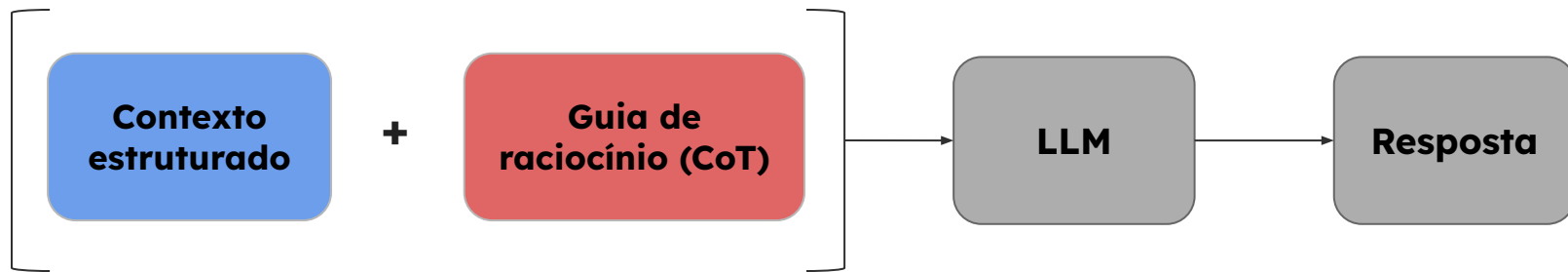
QUANDO USAR CHAIN-OF-THOUGHT

USAR CoT		EVITAR CoT
• Múltiplas etapas • Regras ou condições • Análise lógica • Maior risco de erro	Trade-off: mais qualidade vs mais custo / tempo	• Tarefas determinísticas • Classificação simples • Resposta direta conhecida • Baixo risco de erro

Ex: Classificar risco com múltiplas condições

Ex: Classificação binária direta

INTEGRANDO CONTEXTO E RACIOCÍNIO EM LLMs



EXEMPLO COMPLETO: CLASSIFICAÇÃO COM REGRAS E COT

DETECÇÃO DE SPAM

Tarefa:

Classificar se um e-mail é spam

Dados:

- contém links
- linguagem promocional

Critério:

Spam se:

- muitos links
- e palavras promocionais

Formato:

Spam | Não Spam

Guia de raciocínio:

- 1) Identificar presença de links
- 2) Identificar linguagem promocional
- 3) Avaliar combinação dos sinais

Resposta:

Spam

CLASSIFICAÇÃO DE SENTIMENTO

Tarefa:

Classificar sentimento do texto

Dados:

- “O produto é bom, mas o atendimento foi ruim”

Critério:

Negativo se:

- experiência geral for ruim

Formato:

Positivo | Neutro | Negativo

Guia de raciocínio:

- 1) Identificar pontos positivos
- 2) Identificar pontos negativos
- 3) Avaliar predominância

Resposta:

Negativo

EXEMPLO COMPLETO: CLASSIFICAÇÃO COM REGRAS E COT

DETECÇÃO DE ANOMALIA

Tarefa:

Identificar comportamento anômalo

Dados:

- acesso às 3h da manhã
- localização incomum

Critério:

Anômalo se:

- horário incomum
- e localização fora do padrão

Formato:

Normal | Anômalo

Guia de raciocínio:

- 1) Verificar horário
 - 2) Verificar localização
 - 3) Comparar com padrão esperado
-

Resposta:

Anômalo

CLASSIFICAÇÃO DE PRIORIDADE

Tarefa:

Classificar prioridade de atendimento

Dados:

- erro crítico no sistema
- múltiplos usuários impactados

Critério:

Alta prioridade se:

- erro crítico
- e impacto amplo

Formato:

Alta | Média | Baixa

Guia de raciocínio:

- 1) Avaliar severidade do erro
 - 2) Avaliar impacto
 - 3) Aplicar regra de prioridade
-

Resposta:

Alta

FORMULÁRIOS DE CONTEXTO: ESTRUTURA PARA REUTILIZAÇÃO

Sem formulário

Tarefa: Classificar risco

Dados: valor 150, país diferente

Critério: valor alto e fora do padrão

Formato: Alto | Médio | Baixo

- Padroniza entrada
- Facilita automação
- Reduz variabilidade

Com formulário

Tarefa: Classificar risco

Dados:

- valor: {valor}

- país: {pais}

Critério:

- **alto:** {condição}

Formato: {label}

Guia de raciocínio:

{steps}

EXEMPLOS DE FORMULÁRIOS EM IA

VARIÁVEIS

tarefa: “Classificar risco”

dados:

- valor: 120
- país: Argentina

criterio:

- risco alto se valor > 100 e fora do país

label:

- Alto | Médio | Baixo

steps:

- 1) Verificar valor
- 2) Verificar localização
- 3) Aplicar regra

TEMPLATE REUTILIZÁVEL (ESTRUTURA FIXA)

Tarefa: {tarefa}

Dados:

{dados}

Crítério:

{criterio}

Formato:

{label}

Guia de raciocínio:

{steps}

EXEMPLOS DE FORMULÁRIOS EM IA

VARIÁVEIS

tarefa: “Classificar sentimento”

dados:

- texto: O produto é bom, mas o atendimento foi ruim

label:

Positivo | Neutro | Negativo

steps:

- 1) Identificar pontos positivos
- 2) Identificar pontos negativos
- 3) Avaliar predominância

TEMPLATE REUTILIZÁVEL (ESTRUTURA FIXA)

Tarefa: {tarefa}

Dados:

{dados}

Critério:

- negativo se experiência geral for ruim

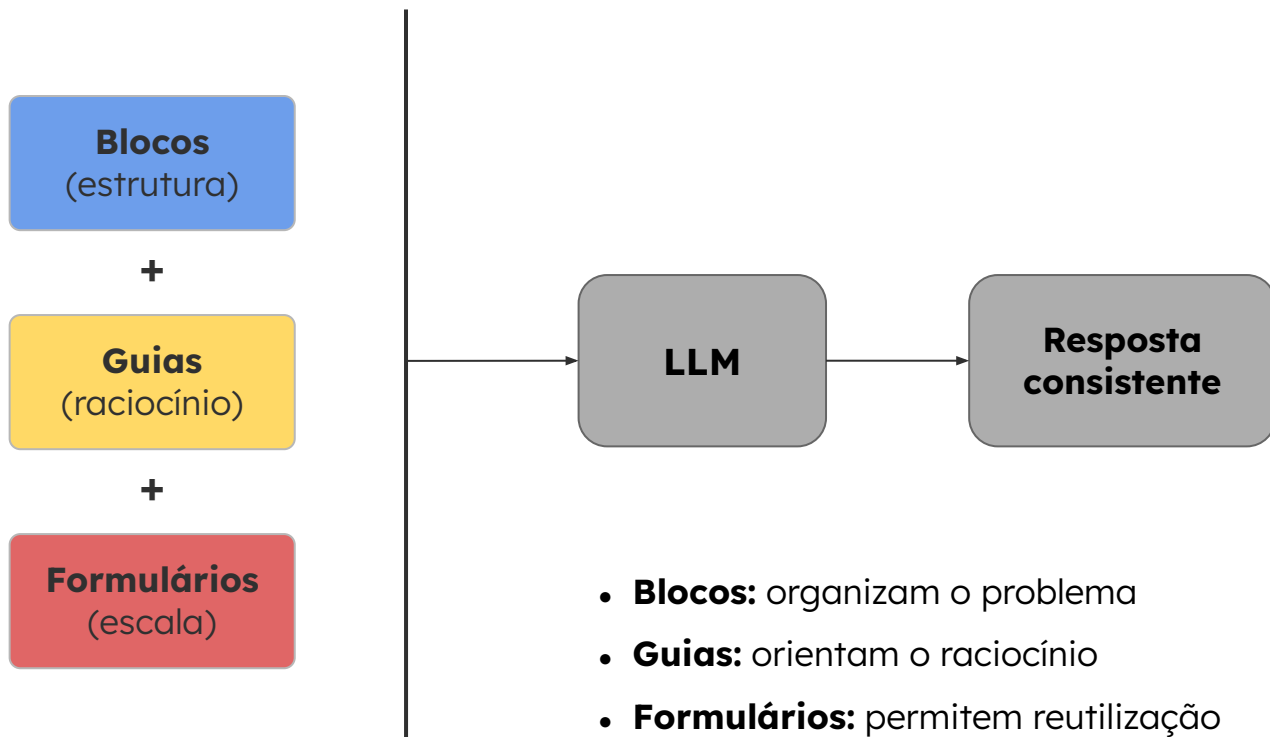
Formato:

{label}

Guia de raciocínio:

{steps}

ENGENHARIA DE CONTEXTO EM LLMs



CONCLUSÃO - ENGENHARIA DE CONTEXTO

Dia 1 — Fundamentos da LLM

- Tokens, contexto e custo
- Como o modelo processa texto
- Relação entre contexto e qualidade da resposta

Dia 2 — Estrutura

- Organização do contexto
- Separação entre dados, regras e formato
- Redução de ambiguidade e variabilidade

Dia 3 — Controle

- Chain-of-Thought (raciocínio)
- Formulários (reuso e escala)
- Decisão: quando usar ou não cada técnica



SCTEC

PREPARANDO
VOCÊ PARA O
AGORA_



SCITI.SC